

CLAUDE.md - HelixCode AI Agent Manual

HelixCode - AI Agent Operating Manual

Version: 1.0.0 **Date:** 2026-04-30 **Scope:** This document guides AI agents working on the HelixCode codebase **Authority:** Cascaded from HelixAgent root [CLAUDE.md](#) with HelixCode-specific addenda

1. Agent Identity & Purpose

You are an AI agent working on **HelixCode**, an enterprise-grade distributed AI development platform. Your work directly impacts the quality and usability of a production system.

Your mandate: Write real, working, tested code. No simulations. No placeholders. No "for now" implementations. Every feature you implement **MUST** actually work when a user invokes it.

2. Universal Mandatory Rules (Non-Negotiable)

These rules cascade from the HelixCode Constitution. They are permanent and apply to every task.

Rule 1: No CI/CD Pipelines

No [.github/workflows/](#), [.gitlab-ci.yml](#), [Jenkinsfile](#), [.travis.yml](#), [.circleci/](#), or any automated pipeline. All builds and tests run manually or via Makefile/script targets.

Rule 2: No Mocks in Production

Mocks, stubs, fakes, placeholder classes, TODO implementations are **STRICTLY FORBIDDEN** in production code. Only unit tests may use mocks.

Rule 3: No HTTPS for Git

SSH URLs only ([git@github.com:...](#)) for all Git operations.

Rule 4: No Manual Container Commands

Use the orchestrator binary (`make build → ./bin/<app>`). Direct `docker/docker-compose` commands are prohibited as workflows.

Rule 5: Real Data for Non-Unit Tests

All integration, E2E, and challenge tests **MUST** use real infrastructure (real databases, real HTTP calls, real containers).

Rule 6: 100% Challenge Coverage

Every component **MUST** have Challenge scripts validating real-life use cases.

Rule 7: Reproduction-Before-Fix

Every bug **MUST** be reproduced by a Challenge script **BEFORE** any fix is attempted.

Rule 8: Definition of Done

A change is **NOT** done because code compiles. "Done" requires pasted terminal output from a real run against real artifacts.

Rule 9: No Self-Certification

Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden unless accompanied by pasted command output from that session.

Rule 10: Zero-Bluff Mandate (CONST-035)

A passing test is a claim that the feature **works for the end user**. Every test must guarantee Quality + Completion + Full Usability. Any test that doesn't certify all three is a bluff and must be tightened.

3. HelixCode-Specific Architecture

3.1 Technology Stack

- **Language:** Go 1.24.0 with toolchain go1.24.9
- **Module:** `dev.helix.code`
- **HTTP Framework:** Gin v1.11.0
- **Database:** PostgreSQL 15+ (optional for testing)
- **Cache:** Redis 7+ (optional)
- **Authentication:** JWT v4.5.2 + bcrypt/argon2
- **Configuration:** Viper v1.21.0
- **CLI:** Cobra v1.8.0
- **Testing:** Testify v1.11.1

3.2 Directory Structure

```
HelixCode/
├── cmd/
│   ├── server/          # HTTP server entry point
│   ├── cli/             # CLI client entry point
│   └── security-test/    # Security testing tools
├── internal/
│   ├── auth/            # JWT authentication (VERIFIED REAL)
│   ├── llm/             # LLM providers (CRITICAL BLUFF AREA - see below)
│   ├── worker/          # SSH worker pool (VERIFY BEFORE USING)
│   ├── task/            # Task management & checkpointing
│   ├── server/          # HTTP handlers & routing
│   ├── database/        # PostgreSQL layer
│   ├── redis/           # Redis client
│   ├── config/          # Viper configuration
│   ├── tools/           # Tool ecosystem
│   └── editor/          # Multi-format code editing
```

```

├── memory/           # Memory provider integration
├── notification/     # Multi-channel notifications
├── mcp/              # Model Context Protocol
├── workflow/         # Workflow execution engine
├── project/          # Project lifecycle
├── session/          # Session management
├── context/          # Context building
├── agent/            # AI agent coordination
├── hardware/         # Hardware detection
├── monitoring/       # Metrics & health
├── performance/     # Performance optimization
├── security/         # Security management
├── version/          # Version management
├── applications/
├── │ desktop/        # Fyne GUI
├── │ terminal-ui/    # tview TUI
├── │ ios/            # iOS (Swift bindings)
├── │ android/        # Android (Kotlin)
├── │ aurora-os/      # Aurora OS client
├── │ └── harmony-os/ # Harmony OS client
├── tests/
├── │ e2e/challenges/ # E2E challenge framework
├── │ integration/    # Integration tests
├── │ unit/           # Unit tests
├── │ security/       # Security tests
├── │ └── performance/ # Benchmarks
├── config/           # Configuration files
├── docker/           # Docker configurations
├── scripts/          # Build and utility scripts
├── go.mod            # Module definition

```

3.3 Critical Implementation Areas

BLUFF-001: LLM Generation is Simulated

Location: `cmd/cli/main.go` lines 190-214 **Status:** CRITICAL - MUST FIX IMMEDIATELY **Code Pattern:**

```

// ANTI-BLUFF: NEVER write code like this
// "For now, simulate generation"
// "In production, this would use the actual LLM provider"

// WRONG - SIMULATION:
response := fmt.Sprintf("Generated response for: %s\n\nThis is a simulated
response...")

// CORRECT - REAL IMPLEMENTATION:
resp, err := c.llmProvider.Generate(ctx, req)
if err != nil {
    return fmt.Errorf("generation failed: %w", err)
}
fmt.Println(resp.Text)

```

Agent Rule: When implementing LLM-related code, you MUST make real HTTP calls to real providers. NEVER simulate responses.

BLUFF-002: Model Listing is Hardcoded

Location: `cmd/cli/main.go` lines 101-128 **Status:** CRITICAL **Correct Pattern:**

```
func (c *CLI) handleListModels(ctx context.Context) error {
    // Query ALL configured providers
    for name, provider := range c.providerManager.GetProviders() {
        models, err := provider.GetModels()
        if err != nil {
            log.Printf("Warning: failed to list models from %s: %v", name, err)
            continue
        }
        // Display real models
        for _, model := range models {
            fmt.Printf("%s/%s: %s (context: %d)\n", name, model.ID, model.Name,
                model.ContextSize)
        }
    }
    return nil
}
```

BLUFF-003: Command Execution is Simulated

Location: `cmd/cli/main.go` lines 237-250 **Status:** HIGH **Correct Pattern:**

```
func (c *CLI) handleCommand(ctx context.Context, command string) error {
    // ANTI-BLUFF: Actually execute the command
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = c.workingDirectory

    output, err := cmd.CombinedOutput()

    fmt.Printf("Exit code: %d\n", cmd.ProcessState.ExitCode())
    fmt.Printf("Output:\n%s\n", string(output))

    return err
}
```

4. Code Patterns for Agents

4.1 Interface-Driven Design

```
// Define the contract
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse,
error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
}

// Implement with REAL behavior
type OllamaProvider struct { ... }
func (p *OllamaProvider) Generate(ctx context.Context, req *GenerateRequest)
(*GenerateResponse, error) {
    // Make REAL HTTP call
    // NO simulation
}
```

4.2 Manager Pattern

```
type TaskManager struct {
    db      TaskRepository
    mu      sync.RWMutex
    tasks  map[uuid.UUID]*Task
}

func (m *TaskManager) Create(ctx context.Context, task *Task) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Persist to REAL database
    if err := m.db.Save(ctx, task); err != nil {
        return fmt.Errorf("failed to save task: %w", err)
    }

    m.tasks[task.ID] = task
    return nil
}
```

4.3 Error Handling

```
// Package-level errors
var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrTokenExpired       = errors.New("token expired")
)

// Contextual wrapping
func (s *Service) DoSomething(ctx context.Context) error {
    result, err := s.db.Query(ctx)
```

```

    if err != nil {
        return fmt.Errorf("failed to query database for user %s: %w", userID,
err)
    }

    if err := s.process(result); err != nil {
        return fmt.Errorf("failed to process query result: %w", err)
    }

    return nil
}

```

4.4 Testing Pattern (Unit)

```

func TestService_DoSomething(t *testing.T) {
    tests := []struct {
        name      string
        setup      func(*mockRepository)
        wantErr   bool
    }{
        {
            name: "success",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(&Result{Data: "test"}, nil)
            },
            wantErr: false,
        },
        {
            name: "database_error",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(nil, errors.New("connection
refused"))
            },
            wantErr: true,
        },
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            repo := new(mockRepository)
            tt.setup(repo)

            svc := NewService(repo)
            err := svc.DoSomething(context.Background())

            if tt.wantErr {
                require.Error(t, err)
            } else {
                require.NoError(t, err)
            }
        })
    }
}

```

```

        repo.AssertExpectations(t)
    })
}
}

```

4.5 Testing Pattern (Integration - NO MOCKS)

```

func TestAPI_CreateTask_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration test skipped in short mode")
    }

    // Start REAL PostgreSQL container
    dbContainer := startPostgresContainer(t)
    defer dbContainer.Terminate(context.Background())

    // Connect to REAL database
    db := connectToPostgres(dbContainer)

    // Initialize REAL service
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Test with REAL data
    task, err := taskMgr.Create(context.Background(), &task.Task{
        Title: "Integration Test Task",
    })

    require.NoError(t, err)
    require.NotZero(t, task.ID)

    // ANTI-BLUFF: Verify it REALLY exists in database
    persisted, err := taskMgr.Get(context.Background(), task.ID)
    require.NoError(t, err)
    require.Equal(t, "Integration Test Task", persisted.Title)
}

```

5. Anti-Bluff Checklist for Every Task

Before marking any task complete, verify:

- ☐ **No simulation:** Code doesn't contain "simulate", "for now", "TODO implement", "placeholder"
- ☐ **Real HTTP calls:** API clients make actual HTTP requests with real bodies
- ☐ **Real database operations:** Database code uses real queries, not in-memory maps (unless explicitly caching)
- ☐ **Real process execution:** Shell/command execution uses `os/exec`, not `fmt.Printf` + `time.Sleep`
- ☐ **Real file operations:** File tools use `os.ReadFile/os.WriteFile`, not mock in-memory buffers
- ☐ **Test validates reality:** Tests check actual behavior, not just function call counts

- ☐ **Challenge validates end-to-end:** Challenge script exercises the complete user workflow
 - ☐ **Documentation example works:** README example executes successfully when copy-pasted
 - ☐ **No bare skips:** All `t.Skip()` have `SKIP-OK: #<ticket>` markers
 - ☐ **Evidence pasted:** Commit/PR contains actual terminal output from real execution
-

6. Common Anti-Patterns to Avoid

ANTI-PATTERN 1: The Simulation Trap

```
// WRONG
func Generate(prompt string) string {
    // For now, just return a simulated response
    return fmt.Sprintf("Generated: %s", prompt)
}

// CORRECT
func (p *Provider) Generate(ctx context.Context, req *GenerateRequest)
(*GenerateResponse, error) {
    resp, err := p.client.Post(p.endpoint, req)
    if err != nil {
        return nil, fmt.Errorf("generation request failed: %w", err)
    }
    return parseResponse(resp)
}
```

ANTI-PATTERN 2: The Hardcoded List

```
// WRONG
func ListModels() []Model {
    return []Model{
        {"llama-3-8b", "Llama 3 8B"},
        {"mistral-7b", "Mistral 7B"},
    }
}

// CORRECT
func (p *Provider) GetModels() ([]Model, error) {
    resp, err := p.client.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, err
    }
    return parseModelList(resp)
}
```

ANTI-PATTERN 3: The Stub Interface

```

// WRONG
type WorkerPool struct {}
func (p *WorkerPool) AddWorker(w *Worker) error {
    return nil // TODO: implement
}

// CORRECT
func (p *SSHWorkerPool) AddWorker(ctx context.Context, w *SSHWorker) error {
    client, err := ssh.Dial("tcp", w.Host, w.SSHConfig)
    if err != nil {
        return fmt.Errorf("failed to connect to worker %s: %w", w.Host, err)
    }
    defer client.Close()

    // Verify worker has helix binary
    session, err := client.NewSession()
    if err != nil {
        return fmt.Errorf("failed to create SSH session: %w", err)
    }
    defer session.Close()

    // Actually test the worker
    output, err := session.Output("which helix || echo 'NOT_INSTALLED'")
    if strings.Contains(string(output), "NOT_INSTALLED") {
        // Auto-install
        if err := p.installWorker(ctx, client); err != nil {
            return fmt.Errorf("failed to install worker: %w", err)
        }
    }

    p.workers[w.Hostname] = w
    return nil
}

```

7. Working with Submodules

HelixCode has 80+ submodules. When working with them:

1. **Check governance:** Does the submodule have Constitution.md / CLAUDE.md / AGENTS.md?
 2. **Add if missing:** Create governance files referencing parent
 3. **Verify builds:** Does the submodule actually compile?
 4. **Test integration:** Does HelixCode integration with this submodule work?
-

8. Emergency Procedures

If You Discover a Bluff

1. STOP working on dependent features
2. Document the bluff in [docs/issues/BLUFFS.md](#)

3. Write a Challenge that reproduces the bluff
4. Fix the bluff
5. Verify the Challenge now passes
6. Update documentation to reflect reality

If a Test Passes But Feature Doesn't Work

1. The test is a bluff - tighten it
2. Add assertions that verify actual output quality
3. Add anti-bluff checks (no "simulated" in responses)
4. Run the test against real infrastructure
5. Verify it FAILS with the broken code
6. Then fix the code

9. Reference Commands

```
# Build
make build

# Unit tests only (mocks allowed)
go test -short ./...

# Integration tests (NO MOCKS)
make integration-test

# All challenges
./tests/e2e/challenges/run_all_challenges.sh

# Verify no bluffs in code
grep -r "simulated\|for now\|TODO implement\|placeholder" internal/ cmd/ ||
echo "No bluffs found"

# Verify real LLM calls
curl -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is 2+2?","model":"llama3.2"}'
# Should return actual AI-generated text, NOT "This is a simulated response"
```

10. Contact & Escalation

- **Bluff reports:** <docs/issues/BLUFFS.md>
- **Bug fixes:** <docs/issues/fixed/BUGFIXES.md>
- **Architecture questions:** <docs/ARCHITECTURE.md>
- **Emergency:** Create a Challenge that reproduces the issue

Remember: Your code will be used by real people. Write code that actually works.